



THE COMPLETE GUIDE · 2026 EDITION

Artificial Intelligence in Practice

Local Models, RAG, Agents and Intelligent Automation

Ollama · Qwen · Llama · CrewAI · LangGraph · MCP · Claude

BY

Vinicius Pereira

Notice and terms of use

This material is educational. Its goal is to teach the concepts and workflows of applied Artificial Intelligence, with practical examples. It was written to serve as a study foundation before hands-on work.

The field of AI moves fast. Prices, software versions, model names and commands can change at any time. Every figure in this edition was verified in 2026, but you should always confirm values and commands against official sources before making purchasing decisions or putting anything into production.

WARNING — Before using in production

The code examples are didactic and illustrate the logic of each tool. Before using them professionally, test every snippet in your own environment — libraries like CrewAI and LangChain change their syntax frequently.

Nothing in this material constitutes legal, financial or tax advice. Payroll, tax and legal calculations must always be validated by a qualified professional.

Product names, brands and companies mentioned belong to their respective owners and are referenced for educational and reference purposes only.

Table of Contents

Notice and terms of use.....	1
The progressive structure.....	5
Visual conventions.....	5
The code blocks.....	5
AI, Machine Learning and LLMs: fitting the pieces.....	8
Why it looks like magic (but isn't).....	8
Open vs closed models.....	9
Token: the currency of models.....	10
Context: the short-term memory.....	10
Temperature: the creativity knob.....	10
The model's parameters (the "B").....	11
Putting it together in an example.....	11
Editor vs IDE: the difference.....	13
VS Code: the industry standard.....	13
The alternatives and when they make sense.....	13
Cursor: is it worth the price?.....	14
The terminal, without fear.....	15
Python and why the virtual environment.....	15
Terminal commands you'll always use.....	16
What Claude Code is.....	17
Editor integration.....	17
Why run locally.....	20
Installing Ollama.....	20
The essential commands.....	20
Measuring speed: tokens per second.....	21
Seeing the context size.....	21
The 2026 landscape.....	22
Families and their vocations.....	22
Recommendations by task.....	22
The core idea.....	24
The levels in practice.....	24
Calculating whether a model fits.....	24
PCs with a dedicated GPU.....	26
Apple Silicon: unified memory.....	26
Checking the optimization.....	27
Open WebUI.....	28
LM Studio.....	28
The problem RAG solves.....	30
How it works, step by step.....	30
Embeddings: turning meaning into numbers.....	32
Vector databases: where the vectors live.....	32
AnythingLLM.....	33
Other visual options.....	33
Installing the dependencies.....	34
The complete pipeline.....	34
The art of chunking.....	36

Quality of the source documents.....	36
Reducing hallucinations.....	36
When RAG isn't the right answer.....	36
Chat vs Agent.....	38
The agent's cycle.....	38
What an agent can do for you.....	39
The CrewAI philosophy.....	40
Anatomy of an agent.....	40
Tasks and assembling the crew.....	41
Where tools come from.....	42
Accessing files on the computer.....	42
Browsing the internet.....	42
Controlling a real browser.....	43
Turning code into a tool.....	44
The description is everything.....	44
How agents communicate.....	45
Sequential vs hierarchical mode.....	45
Flows: CrewAI's flow control.....	46
LangChain: the Swiss Army knife.....	48
LangGraph: flows as graphs.....	48
CrewAI vs LangGraph: which to choose.....	48
LangSmith: seeing what the agent thinks.....	49
The problem MCP solves.....	50
The architecture, in simple terms.....	50
Why this matters to you.....	50
Agents that browse the PC and the web via MCP.....	51
Two protocols, two roles.....	52
Agents from different frameworks working together.....	52
When several agents are worth it.....	53
The idea.....	55
The decisive question: RAG or fine-tuning?.....	55
Why LoRA changed the game.....	56
Tools for fine-tuning.....	56
The dataset is what matters.....	56
The two cost models.....	58
Anthropic API prices (verified in 2026).....	58
The hybrid strategy.....	58
The question that defines everything: where does the model run?.....	60
The recommended combination to start.....	60
The real risks of autonomous agents.....	61
Defense principles.....	61
The privacy advantage of local.....	61
Why the prompt matters so much.....	63
The four pillars of a good prompt.....	63
The system prompt.....	64
Iteration: the secret to good results.....	64
Few-shot: teaching by example.....	65
Chain-of-thought: think before answering.....	65
Structured output (JSON).....	65

Decomposition: breaking down the problem.....	66
First steps.....	67
A basic call.....	67
Connecting Claude to a CrewAI agent.....	67
System prompt in the API.....	68
The problem with eyeballing it.....	69
What to measure.....	69
The "AI judge" technique.....	69
Observability tools.....	70
The goal.....	72
The architecture.....	72
Quick version (no code).....	72
Programmable version.....	72
The goal.....	74
The architecture.....	74
The skeleton in CrewAI.....	74
The goal.....	76
The architecture.....	76
From project to product.....	77

CHAPTER 0

How to use this book

This is not a book to be merely read — it is a manual to be executed. Passive reading teaches the vocabulary; practice teaches the craft.

The progressive structure

The content is organized into eleven parts that move from the most basic concept to a complete project. If you are starting from zero, read in order. If you already have experience, use the table of contents to jump straight to what you need — each chapter is reasonably self-contained.

If you are...	Start with	Focus
A complete beginner	Parts I and II	Understand concepts and set up the environment
A programmer new to AI	Parts III and IV	Run models and build RAG
Already into AI	Parts V, VI and VII	Agents, orchestration and fine-tuning
After a quick result	Part X	Copy a ready-made project and adapt it

Visual conventions

Throughout the book you will find highlighted boxes that signal different kinds of information:

KEY CONCEPT — Key concept

Fundamental ideas worth memorizing. They appear when a concept is the foundation for everything that follows.

TIP — Tip

Practical shortcuts, best practices and tricks that save time and avoid headaches.

WARNING — Warning

Common pitfalls, frequent mistakes and the spots where it is easy to get hurt.

COST — Cost

Whenever there is a financial implication — API prices, hardware costs, spending comparisons.

The code blocks

Commands and code appear in dark blocks, like this one:

```
# This is a terminal command
```

```
ollama run qwen2.5-coder:14b
```

You can type them exactly as shown. Lines starting with # are comments — explanations the computer ignores.

NOTE

Technical terms are highlighted the first time they appear and are gathered in the Glossary at the end of the book. If you hit something unfamiliar, look there.

PART I

Foundations

What AI is, how models think, and why that matters to you

Before installing tools or writing code, let's build the right intuition. Everything that comes later — local models, RAG, agents — gets simpler once these concepts are solid.

CHAPTER 1

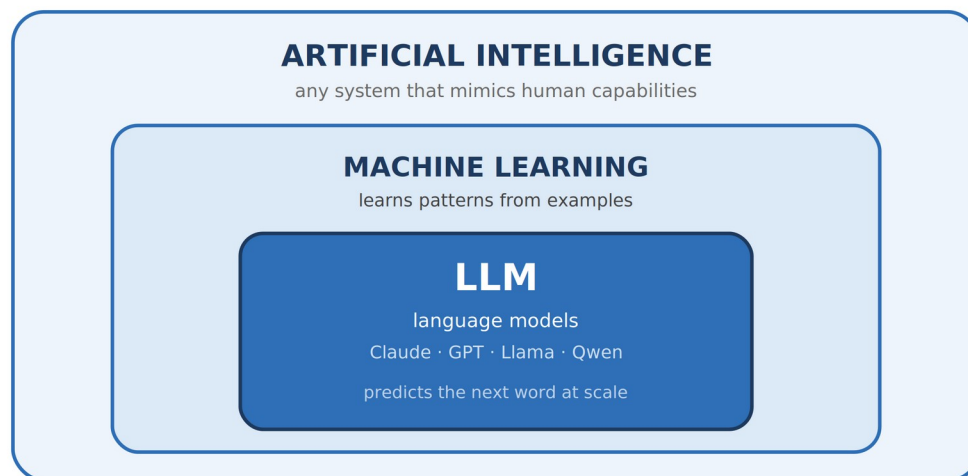
What Artificial Intelligence really is

Before installing anything, it pays to understand what's happening under the hood. You don't need math — you need good intuition. That's what this chapter delivers.

AI, Machine Learning and LLMs: fitting the pieces

The three terms appear together all the time, but they mean different things, nested inside one another like Russian dolls.

- **Artificial Intelligence (AI):** the umbrella field. Any system that performs tasks that would normally require human intelligence.
- **Machine Learning (ML):** a branch of AI where the system learns patterns from examples, instead of following hand-written rules.
- **Large Language Models (LLMs):** a specific type of ML, trained on enormous amounts of text to predict and generate language. Claude, GPT, Llama and Qwen are LLMs.



LLMs are a type of Machine Learning, which is in turn a branch of Artificial Intelligence.

KEY CONCEPT — The essence of an LLM

At its core, an LLM is a next-word prediction machine. You give it a stretch of text and it computes the most likely continuation, word by word. All the apparent intelligence emerges from that simple task repeated at massive scale.

Why it looks like magic (but isn't)

When a model has been trained reading a huge fraction of everything humanity has ever written — books, code, articles, conversations — predicting the next word accurately requires, in practice, having

captured patterns of grammar, logic, facts and even styles of reasoning. The model has no database of answers: it has a web of statistical relationships rich enough to produce coherent, useful text about almost anything.

Understanding this has an important practical consequence: the model can be confidently wrong. Because it generates what is likely, not what is true, it sometimes makes things up — a phenomenon known as hallucination. Hold on to that; we'll return to it several times.

WARNING — Hallucination

When a model states something false with the appearance of truth. It is inherent to how LLMs work. Techniques like RAG (Part IV) exist largely to reduce this problem.

Open vs closed models

There are two broad families of models, and the difference shapes everything you can do with them.

	Closed models	Open models
Examples	Claude, GPT, Gemini	Llama, Qwen, Gemma, DeepSeek
Where they run	The company's servers (cloud)	On your computer or server
Access	Via API or app, pay per use	Free download of the weights
Privacy	Data passes through the company	Data stays with you
Top-tier quality	Generally superior	Excellent and improving fast
Customization	Limited	Total (free fine-tuning)

This book works with both worlds and, more importantly, teaches you to combine them — using open local models for routine work and top-tier closed models when the task demands maximum quality.

CHAPTER 2

Tokens, context and the knobs you control

To use AI well, you need to understand four concepts: token, context, temperature and parameters. They show up in every tool, every price and every setting.

Token: the currency of models

Models don't see words — they see tokens, pieces of text. A token is on average about three-quarters of a word in English. The word "automation" might become two or three tokens; "house" is probably a single one.

KEY CONCEPT — Why tokens matter

Everything is measured and billed in tokens: the size of what you send (input), the size of the response (output) and how much fits in the model's memory. When you pay for an API, you pay per token. When a model gets slow, it's because it's processing many tokens.

A useful rule of thumb: about 750 words equal roughly 1,000 tokens. A page of dense text lands around 500 to 600 tokens.

Context: the short-term memory

Context is the amount of tokens the model can "see" at once — everything in the current conversation, summing your question, the history and the documents you attached. It's like the model's working memory.

When you hear "1M of context", it means the model accepts up to a million tokens at once — the equivalent of several books. Local models usually have a much smaller context by default (4,000 to 32,000 tokens), and this is configurable.

WARNING — The danger of a full context

When the conversation fills the context, the model starts to "forget" the beginning — old information leaves the window. In long projects with local models, this explains why it sometimes loses the thread. Managing context is a core skill, especially with agents.

Temperature: the creativity knob

Temperature controls how predictable or creative the response is. It's a number, usually between 0 and 1 (sometimes up to 2).

Temperature	Behavior	When to use
0 – 0.3	Precise, deterministic, repeatable	Code, data, calculations, extraction
0.4 – 0.7	Balanced	General conversation,

Temperature	Behavior	When to use
		explanations
0.8 – 1.2	Creative, varied, surprising	Brainstorming, writing, ideas

TIP

For business tasks — calculating, extracting, classifying, following rules — use low temperature. You want the same correct answer every time, not creative variations.

The model's parameters (the "B")

When you see a model called "14B" or "70B", the B stands for billions of parameters — the adjustable values the model learned during training. Broadly, more parameters mean more capability, but also more memory and processing.

Don't confuse parameters with size on disk. A 14B model doesn't take up 14GB — it depends on quantization (Chapter 8). In Q4, a 14B model takes about 9GB. Parameters measure the size of the brain; quantization decides how much it is compressed to fit your machine.

Putting it together in an example

Imagine you send a model: "Summarize this 10-page contract into 5 bullet points."

1. **Input:** the 10 pages (~5,500 tokens) plus your instruction enter the context.
2. **Processing:** the model reads everything inside the context window and computes the answer.
3. **Output:** the 5 bullet points (~200 tokens) are generated, one token at a time.
4. **Cost (if API):** you pay for the ~5,700 input tokens and ~200 output tokens.

With these four concepts — token, context, temperature and parameters — you already understand the vocabulary that appears in any tool, price or tutorial from here on.

PART II

The Working Environment

Editors, the terminal and the tools where the work happens

With the concepts in place, it's time to prepare the workshop. This part sets up the environment you'll use in every project in the book — and takes the fear out of the terminal.

CHAPTER 3

Code editors and IDEs

All hands-on work with applied AI happens inside an editor. Choosing the right one and configuring it well is the first investment that pays off in productivity.

Editor vs IDE: the difference

A code editor is a program for writing and editing code, with features that understand the language. An IDE (Integrated Development Environment) is a complete package that brings together editor, run tools, debugger and project manager in one place.

NOTE — Analogy

If programming were construction, the editor would be the drafting table. The IDE would be the whole office — drafting table, calculator, ruler and filing cabinet together. In modern practice, the line between the two is thin: a good editor with extensions becomes practically an IDE.

VS Code: the industry standard

Visual Studio Code, from Microsoft, is free, open source and runs on Windows, macOS and Linux. It's the most-used editor in the world for a reason: it found the balance between lightness and power.

The features that matter

- **IntelliSense** — smart autocomplete that suggests code as you type.
- **Extensions** — a store with thousands of plugins: Python support, themes, Git, Docker, and practically any technology.
- **Integrated terminal** — you run commands without leaving the editor. Essential for everything in this book.
- **Debugger** — pauses running code to inspect variables and understand bugs.
- **Git integration** — visual version control, seeing exactly what changed in the code.

TIP — Essential extensions to start

For Python: Microsoft's official "Python" extension. To live with AI: "Continue" (connects local and cloud models to the editor). For looks: themes like "Catppuccin" or "Tokyo Night". Install them from the extensions tab (the blocks icon on the side).

The alternatives and when they make sense

Editor	Highlight	Ideal for
Cursor	VS Code fork with deep native AI	Those who want AI built in
Zed	Extremely fast, performance-focused	Those who prioritize speed

Editor	Highlight	Ideal for
Neovim / Vim	Runs in the terminal, light and powerful	Advanced users
Sublime Text	Light and fast (paid)	Quick edits
JetBrains (PyCharm)	Robust IDE for large projects	Complex Python projects

Cursor: is it worth the price?

Cursor deserves a mention as the most talked-about AI editor. Because it's based on VS Code, you feel at home, but with AI built into every corner. The plans, verified in 2026:

- **Hobby (free):** limited completions and requests — good for trying it out.
- **Pro (around US\$ 20/month):** unlimited completions, access to top-tier models and a monthly credit pool for premium models.
- **Pro+ and Teams:** more credits and centralized billing for teams.

COST — The credit system

Since mid-2025, Cursor adopted a credit model: using premium models consumes your balance. This drew complaints about unexpected charges. The closest free alternative is VS Code with the Continue extension or Claude Code, paying only for the model you use.

CHAPTER 4

Terminal and the Python environment

The terminal is scary at first, but it's where the magic happens. Mastering half a dozen commands and the concept of a virtual environment already puts you ahead of most.

The terminal, without fear

The terminal is simply a way to give orders to the computer by text, instead of clicks. On macOS and Linux it comes pre-installed; on Windows, the recommended setup is Windows Terminal with WSL (a Linux inside Windows). In VS Code there's an integrated terminal — open it with Ctrl+` (the backtick key).

Those dark, stylish screens you see in posts around the internet are simply terminals with a nice theme and a stylized font. The terminal itself is the same; only the appearance changes. Tools like iTerm2 (Mac) and themes like Catppuccin produce that look.

Python and why the virtual environment

Almost everything in this book runs on Python. Before installing libraries, you need to understand a concept that prevents 90% of headaches: the virtual environment.

KEY CONCEPT — Virtual environment

An isolated "box" where you install a project's libraries without affecting others. Without it, different projects fight over different versions of the same library, and everything breaks. Create one for each project — always.

Creating and activating an environment

```
# Create the environment (do this once per project)
python -m venv .venv

# Activate on Linux or macOS
source .venv/bin/activate

# Activate on Windows
.venv\Scripts\activate

# When active, the name (.venv) appears at the start of the line
# Now install whatever you want, isolated:
pip install crewai langchain
```

TIP — uv: the modern alternative

Newer tools like CrewAI already use uv (from Astral) as the default manager — it's much faster than pip

and creates environments automatically. Install it with: `pip install uv`. Worth learning; it's becoming the standard.

Terminal commands you'll always use

Command	What it does
<code>cd folder</code>	Enters a folder (change directory)
<code>cd ..</code>	Goes up one folder level
<code>ls</code> (dir on Windows)	Lists files in the current folder
<code>pwd</code>	Shows which folder you're in
<code>mkdir name</code>	Creates a folder
<code>python file.py</code>	Runs a Python script
<code>pip install name</code>	Installs a Python library
<code>clear</code> (cls on Windows)	Clears the terminal screen

CHAPTER 5

Claude Code and AI inside the editor

There's a category of tool that turns the editor into an autonomous programming partner. Claude Code is the most advanced example, and it's worth understanding how it fits.

What Claude Code is

Claude Code is Anthropic's command-line coding tool. You point it at a project folder and it reads the code, understands the structure, and can create, edit and delete files, run commands and fix bugs autonomously. That screen with a pixel-art mascot and the model indicator (for example, "Opus" with a 1M context) that circulates on social media is exactly this tool running in the terminal.

Installation

Anthropic's recommended method is the native installer, which requires no Node.js and updates itself. There's also npm installation (requires Node.js 18 or higher):

```
# Via npm (requires Node.js 18+)
npm install -g @anthropic-ai/claude-code

# Start inside your project folder
cd my-project
claude

# Check version / diagnose
claude --version
claude doctor
```

WARNING — Never use sudo with npm

Installing global npm packages with sudo creates permission problems that are hard to undo. If a permission error appears (EACCES), configure your own directory: `npm config set prefix '~/.npm-global'`.

COST — How Claude Code is billed

It uses a Claude plan (Pro at around US\$ 20/month, or Max) or an API account billed per token. It's separate from the tools you build with local models — here the processing runs on Anthropic's servers.

Editor integration

Claude Code integrates with the main editors. In VS Code, running the claude command in the integrated terminal installs and activates the extension automatically. In JetBrains, there's a dedicated plugin with interactive diff and diagnostics. Cursor, being based on VS Code, also supports it.

TIP — Plan mode

Claude Code has a "plan mode" (toggle with Shift+Tab) where it plans the whole approach before

touching the code. For important tasks, always review the plan before letting it execute — it's the difference between supervising and hoping.

PART III

Local Artificial Intelligence

Running models on your own computer — privacy, zero cost and total control

Environment ready, let's get an AI running on your machine. From here on you stop depending on external services for day-to-day work — and keep your data at home.

CHAPTER 6

Ollama: your first local model

Running an AI on your own machine is no longer a researcher's thing. With one tool and one command, you have a top-tier model answering offline, with no per-use cost.

Why run locally

- **Total privacy** — your data never leaves the computer. For payroll, contracts and personal IDs, that's decisive.
- **Zero cost per use** — running a thousand times costs the same as running once: nothing beyond electricity.
- **Offline** — works without internet, anywhere.
- **No limits** — no message cap, no queue.

Installing Ollama

Ollama is the standard tool for running open models locally. It's free and handles all the heavy lifting of downloading, loading and serving models. The current 2026 version already brings broad hardware support and an engine optimized for Apple Silicon.

```
# Linux
curl -fsSL https://ollama.com/install.sh | sh

# macOS (via Homebrew)
brew install ollama

# Windows: download the installer at ollama.com
```

The essential commands

```
ollama run qwen2.5-coder:14b # downloads (1st time) and starts the chat
ollama pull gemma3:12b     # only downloads, doesn't start
ollama list                 # installed models
ollama ps                  # what's running and on which hardware
ollama show qwen2.5-coder:14b # the model's spec sheet
ollama rm gemma3:12b       # removes a model from disk
```

When you run `ollama run`, it downloads the model the first time and opens a chat right in the terminal. Type your question and press Enter. To exit, type `/bye`.

Measuring speed: tokens per second

The metric that defines the experience is tokens per second (tok/s): how many tokens the model generates each second. To see it, use the `--verbose` flag:

```
ollama run qwen2.5-coder:14b --verbose
```

At the end of each response, Ollama shows statistics. The two that matter:

- **eval rate** — generation tok/s. It's the speed you feel as the answer appears.
- **prompt eval rate** — tok/s while processing what you sent. Matters on long prompts.

KEY CONCEPT — What tok/s reveals

That number is the real performance of YOUR hardware. It's effectively a free benchmark. It reflects your GPU, your memory and the model size — not a theoretical value.

Speed	Usage experience
below 5 tok/s	Slow — usable, but sluggish
10 to 20 tok/s	Comfortable — human reading pace
30 to 60 tok/s	Fast — almost immediate response
80+ tok/s	Very fast — high-end dedicated GPU

Seeing the context size

To find out how much context a model supports, use:

```
ollama show qwen2.5-coder:14b
# Look for 'context length' in the output

# To view/adjust parameters like num_ctx:
ollama show qwen2.5-coder:14b --modelfile
```

TIP — Increasing the context

Many models run with a default context smaller than the maximum, to save memory. You can increase it by creating a Modelfile with `PARAMETER num_ctx 32768` — but remember: more context uses more memory.

CHAPTER 7

Choosing the right model (2026 catalog)

There are more than a hundred models available. The good news: you only need to know a handful. The choice depends on two things — what you're going to do and how much memory you have.

The 2026 landscape

Open models took a huge leap. Families like Qwen, Gemma, Llama and DeepSeek today handle most everyday code and text tasks with quality comparable to cloud models from a generation ago. New names — Qwen 3.x, Gemma 4, Llama 4 Scout, DeepSeek V4 and reasoning models derived from DeepSeek-R1 — fight for the top spot every month.

NOTE — Names change, the logic doesn't

Specific versions age fast. What doesn't change is the selection method: define your task, see how much memory you have, and grab the largest model from the right family that fits with room to spare. The commands below use names valid in 2026; adjust as new versions arrive.

Families and their vocations

Family	Origin	Shines at
Qwen / Qwen-Coder	Alibaba	Code and general tasks — great value
Llama	Meta	General use, largest ecosystem and support
Gemma	Google	High quality, good with tools and vision
DeepSeek (R1 / V4)	DeepSeek	Step-by-step reasoning and debugging
Phi	Microsoft	Logic and math in small models
gpt-oss	OpenAI	Open generalist model, good on 16GB

Recommendations by task

For programming

The Qwen-Coder family is the reference for local code models. Grab the largest size your machine can handle: 32B on a workstation, 14B on a good laptop, 7B on modest hardware.

```
ollama run qwen2.5-coder:14b # excellent on 16GB+ laptops
```

For reasoning and debugging

Reasoning models "think" before answering, showing a chain of thought. The DeepSeek-R1 derivatives are strong at finding subtle bugs and solving logic.

```
ollama run deepseek-r1:14b
```

For general use and tools

Gemma and Llama balance quality and support. Gemma is especially good at calling tools (useful for agents) and handling images.

```
ollama run gemma3:12b  
ollama run llama3.1:8b
```

TIP — Start small, then scale up

Download a 7-8B model first to test that everything works and feel the speed. Only then invest the download time in a larger model. A poorly-chosen 14B that doesn't fit in memory is worse than an 8B that flies.

CHAPTER 8

Quantization: compressing without destroying

It's the concept that decides whether a model fits on your machine or not. Understanding quantization is understanding why a "14B" model can take up 9GB or 28GB.

The core idea

Quantization is the process of reducing the precision of the numbers that make up the model. The original model stores each parameter at high precision (16 bits). Quantization rewrites those values with fewer bits — 8, 5 or 4 — drastically shrinking the file, with a surprisingly small loss of quality.

KEY CONCEPT — The photo analogy

Quantizing is like saving a photo with a bit more JPEG compression. The file gets much smaller, and to the naked eye you barely notice the difference. Only at extreme zoom does the loss show. For most uses, the compressed version is indistinguishable.

The levels in practice

Level	Relative size	Quality	When to use
Q2 / Q3	Minimum	Noticeably worse	Only if nothing larger fits
Q4 (K_M)	Small	Great	Default — best balance
Q5 / Q6	Medium	Excellent	If you have spare memory
Q8	Large	Nearly identical to original	Maximum local fidelity
FP16	Huge	Original	Research and fine-tuning

TIP — The golden rule

For general use, Q4_K_M is the sweet spot and Ollama's default. As a rule of thumb, a model in Q4 takes up in GB about 60% of its parameter count: a 14B lands around 9GB; a 70B, around 40GB.

Calculating whether a model fits

The calculation that matters: the model (in GB) needs to fit in your video memory (VRAM), with 1 to 2GB of headroom for the context. If it doesn't fit in VRAM, part of it "spills" into regular RAM, and speed collapses.

5. **Take the model's size in Q4** (e.g., 14B ≈ 9GB).
6. **Compare with your VRAM** (e.g., 16GB).
7. **Leave headroom for the context** (9GB + 2GB = 11GB ≤ 16GB → fits well).

WARNING — The sign of spillover

If ollama ps shows something like "50% GPU / 50% CPU", the model didn't fit entirely in VRAM. You'll feel it: speed drops to a third or less. The fix is to use a smaller model or a more aggressive quantization.

CHAPTER 9

Hardware: GPU, Apple Silicon and optimization

The same question — "will this model run well here?" — has different answers for a PC with a graphics card and for a Mac. Understanding both architectures avoids bad purchases and frustration.

PCs with a dedicated GPU

On an NVIDIA or AMD card, what matters for AI is the VRAM (the card's memory) and the memory bandwidth. VRAM decides which model fits; bandwidth decides the speed. Ollama uses the GPU automatically when it detects a compatible one.

GPU	VRAM	Recommended model
RTX 3060 / 4060	8 GB	llama3.1:8b or qwen-coder:7b
RTX 4070 / 5060 Ti	12-16 GB	qwen2.5-coder:14b with room
RTX 4090 / 5090	24-32 GB	27-32B models comfortable
2x 24GB GPUs	48 GB	llama 70B in Q4

Apple Silicon: unified memory

Apple chips (M1 through M5 and their Pro/Max/Ultra variants) have a structural advantage for AI: unified memory. CPU and GPU share the same RAM pool, so all of the Mac's memory is available to the model.

KEY CONCEPT — Why this is powerful

On a PC, a model only runs fast if it fits in the card's VRAM — typically 8 to 24GB. On a Mac with 64GB, the model can use almost all that memory. That's why Macs with lots of RAM run huge models (70B+) that would require very expensive graphics cards on a PC.

The trade-off is bandwidth: even on the Max chips, it tends to be lower than a top-end dedicated GPU. In practice, the Mac wins on capacity (the big model fits) and loses a little on raw speed per token against a top GPU.

Mac	Unified memory	Capacity
M-series base	16-24 GB	Models up to 14B comfortable
M Pro	24-48 GB	27-32B models
M Max	36-128 GB	70B in Q4 (with 64GB+)
M Ultra	128-256 GB	Very large models

Checking the optimization

Always confirm the model is really on the GPU. The `ollama ps` command shows the split. "100% GPU" is ideal. Any share on CPU indicates spillover and lost speed.

```
ollama ps
# Ideal output shows '100% GPU' in the PROCESSOR column
```

TIP — Quick optimizations

Close programs that consume VRAM (games, heavy browsers) before running. Use Q4 quantization. If you need more speed, a well-chosen smaller model usually beats a large one spilling over.

CHAPTER 10

Graphical interfaces for local models

The terminal is powerful, but not always comfortable. These interfaces give you the experience of a ChatGPT, running 100% locally on top of Ollama.

Open WebUI

The most complete and popular interface. ChatGPT-style, with conversation history, document upload (built-in RAG) and easy model switching. It runs via Docker and connects to Ollama automatically.

```
docker run -d -p 3000:80 \  
  --add-host=host.docker.internal:host-gateway \  
  ghcr.io/open-webui/open-webui:main  
  
# Then visit http://localhost:3000 in your browser
```

LM Studio

A desktop application (Windows, Mac, Linux) with a graphical interface to download, run and chat with models — no terminal. Excellent for those who prefer clicking to typing commands, and it brings a visual model catalog that indicates which ones fit your hardware.

TIP — Which to choose

LM Studio if you want zero terminal and an app-like experience. Open WebUI if you want a shareable web interface, with built-in RAG, running on top of the Ollama you already set up. Many people use both.

PART IV

RAG — Giving the AI Memory

How to make a model answer based on YOUR documents, not generic knowledge

You already have a model running. But it doesn't know your company. This part fixes that — teaching the AI to consult your documents before answering.

CHAPTER 11

What RAG is and why it changes everything

A model on its own knows a lot about the world, but nothing about your company. RAG is the bridge: the technique that makes the AI consult your own documents before answering.

The problem RAG solves

An LLM was trained at a fixed point in time and doesn't know your internal documents, your business rules, or information from after its training. Worse: when it doesn't know, it tends to make things up (hallucination). RAG attacks both problems at once.

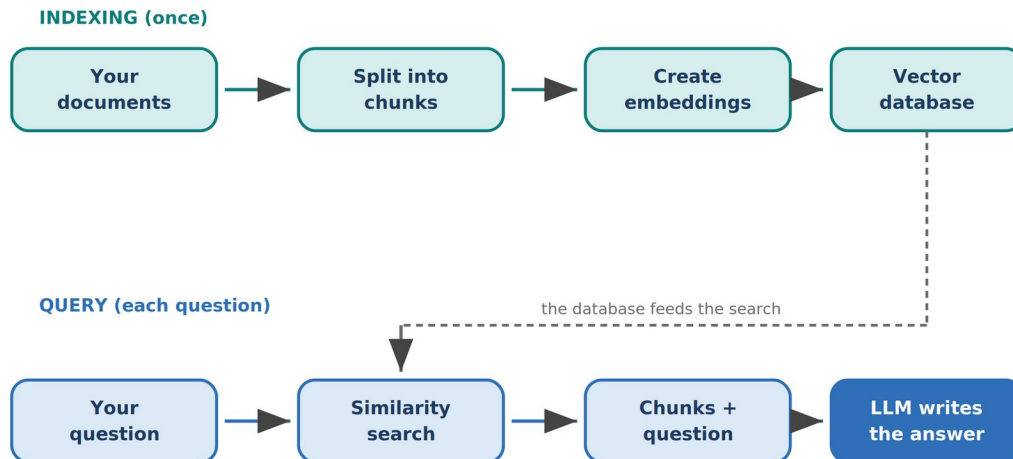
RAG stands for Retrieval Augmented Generation. The name describes the process: before generating the answer, the system retrieves relevant chunks from your documents and hands them to the model along with the question.

KEY CONCEPT — The sentence that sums up RAG

Without RAG, you ask and the model answers off the top of its head. With RAG, the model first consults your documents and answers based on them — citing the source. It's the difference between an intern guessing and a consultant reading the manual before speaking.

How it works, step by step

8. **Indexing (once):** your documents are split into chunks and converted into vectors — numerical representations of meaning. These vectors are stored in a vector database.
9. **Question:** you ask a question, which also becomes a vector.
10. **Search:** the system compares the question's vector with the documents' and finds the most semantically similar chunks.
11. **Assembly:** those chunks are appended to your question, forming an enriched prompt.
12. **Answer:** the model answers using the chunks as the source of truth.



The RAG flow: documents are indexed once; for each question, the system searches the relevant chunks and hands them to the model.

NOTE — Semantic search, not keyword search

The magic is in step 3. The search isn't for matching words, but for meaning. Asking "how much is deducted from retirement?" can retrieve a chunk about "social security rate", even with no words in common, because the vectors capture the sense.

CHAPTER 12

Embeddings and vector databases

These are the two engines underneath RAG. You don't need to implement them from scratch, but understanding what they do completely changes the quality of what you build.

Embeddings: turning meaning into numbers

An embedding is a list of numbers (a vector) that represents the meaning of a text. Texts with similar meanings produce nearby vectors in mathematical space. "Dog" and "hound" sit close together; "dog" and "spreadsheet" sit far apart.

An embedding model is different from a chat LLM — it specializes in producing those vectors. In Ollama, you can run dedicated embedding models, which tend to be small and fast.

```
# Download a dedicated embedding model
ollama pull nomic-embed-text

# It doesn't chat – it only generates vectors for RAG
```

Vector databases: where the vectors live

A vector database stores the embeddings and performs similarity search efficiently, even with millions of chunks. It's the equivalent of a database, but optimized for "find the most similar" instead of "find the exact".

Vector database	Profile
Chroma	Simple, runs locally, ideal to start and for medium projects
FAISS	Meta's library, very fast, great for volume
Qdrant	Robust, good for production, has local and cloud versions
LanceDB	Modern, embedded, no separate server

TIP — To learn, use Chroma

It runs inside your Python script, with no server to stand up, and is more than enough for thousands of documents. Swap it for Qdrant or FAISS only when volume demands.

CHAPTER 13

RAG without writing code

You can have a working RAG over your documents in minutes, without programming a single line. These tools handle all the machinery.

AnythingLLM

The most recommended option to start. It's an app with a complete browser interface that connects directly to Ollama. You create a "workspace", drag in your PDFs, spreadsheets and texts, and start chatting with them. Indexing, embeddings and the vector database are all managed behind the scenes.

Usage flow

13. Install AnythingLLM (anythingllm.com) and open the app.
14. In settings, choose Ollama as the LLM and embedding provider.
15. Create a workspace for your topic (e.g., "Payroll").
16. Drag the relevant documents into the workspace.
17. Ask in the chat — the answers will be based on your files and show the source.

Other visual options

- **Open WebUI** — besides chat, it lets you attach documents in a conversation and enable RAG over them. Good if you already use it for models.
- **Flowise** — builds the RAG pipeline visually, dragging blocks. Excellent for understanding the machinery before moving to code.

KEY CONCEPT — Start with the visual

Even if your final goal is code, building a RAG in AnythingLLM first teaches what each stage does and gives an immediate result to validate the idea. Then migrating to code is just reproducing what you already understand.

CHAPTER 14

RAG with code: total control

When you need to integrate RAG into a system, tune every detail or run at scale, the path is code. Let's build a complete RAG with LangChain and Ollama.

Installing the dependencies

```
# Always inside an active virtual environment
pip install langchain langchain-community chromadb pypdf

# Embedding model in Ollama
ollama pull nomic-embed-text
```

The complete pipeline

This example loads a PDF, indexes its content and answers questions based on it. The comments explain each stage.

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import OllamaEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_community.llms import Ollama
from langchain.chains import RetrievalQA

# 1. Load the document
loader = PyPDFLoader('company_rules.pdf')
documents = loader.load()

# 2. Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,      # ~1000 characters per chunk
    chunk_overlap=150     # overlap so context isn't cut off
)
chunks = splitter.split_documents(documents)

# 3. Generate embeddings and index in the vector database
embeddings = OllamaEmbeddings(model='nomic-embed-text')
db = Chroma.from_documents(chunks, embeddings)

# 4. Assemble the RAG with a local LLM
llm = Ollama(model='qwen2.5-coder:14b')
rag = RetrievalQA.from_chain_type(
    llm=llm,
```

```
    retriever=db.as_retriever(search_kwargs={'k': 4})
)

# 5. Ask
answer = rag.invoke('What is the transit benefit rule?')
print(answer['result'])
```

NOTE — The k parameter

In `search_kwargs={'k': 4}`, `k` defines how many chunks are retrieved for each question. Too few (`k=2`) may lose context; too many (`k=10`) fill the prompt and confuse. Between 3 and 5 is usually the sweet spot.

CHAPTER 15

Best practices that separate a good RAG from a bad one

Building a RAG is easy; building a RAG that answers well takes care with a few details. These are the ones that most impact the result.

The art of chunking

How you split the documents is perhaps the factor that most affects quality. Chunks that are too large bring noise; too small lose context. Overlap prevents a piece of information from being cut exactly at the boundary between two chunks.

- **Running text (contracts, manuals):** chunks of 800 to 1200 characters, with 10-15% overlap.
- **Structured data (tables, FAQs):** split by logical item — one question-answer per chunk, one row per chunk.
- **Code:** split by function or class, not by a fixed character count.

Quality of the source documents

KEY CONCEPT — Garbage in, garbage out

A RAG is only as good as the documents you give it. Scanned PDFs with no searchable text, messy spreadsheets and outdated information produce bad answers. Invest time cleaning and organizing the source — it's what pays off most.

Reducing hallucinations

Even with RAG, the model can extrapolate. Two simple defenses:

18. **Explicit instruction:** in the system prompt, tell it to answer only based on the provided context and to admit when the information isn't there.
19. **Low temperature:** use 0 to 0.2 for factual tasks — you want fidelity to the source, not creativity.

When RAG isn't the right answer

RAG is ideal for factual knowledge and documents. But if your goal is to change the behavior or style of the model — making it always answer in a specific format, or with a particular tone — that's a job for fine-tuning (Part VII), not RAG.

PART V

AI Agents

Beyond chat: AI that makes decisions, uses tools and executes tasks on its own

Up to here, the AI answered what you asked. Now it's going to act. This is the turning point of the book: from a tool that responds to one that executes.

CHAPTER 16

What an agent is

Until now, the AI answered questions. An agent does something fundamentally different: it acts. It plans, uses tools, evaluates results and keeps going until a task is done.

Chat vs Agent

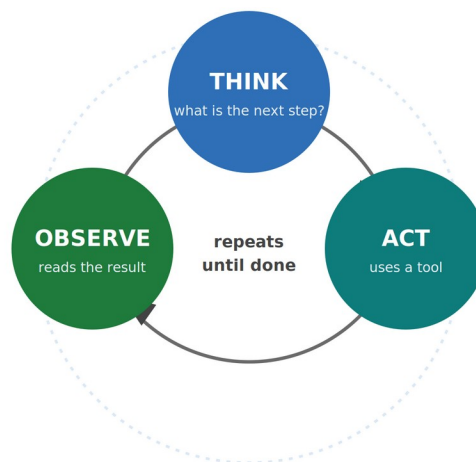
The difference is qualitative, not just one of degree. A chat is reactive — it waits for your next instruction. An agent is proactive — it receives a goal and decides the steps to reach it on its own.

Traditional chat	Agent
Answers one question at a time	Plans and executes several steps
You drive each step	It decides the next steps
Returns only text	Uses tools: files, web, code, APIs
Doesn't check its own work	Evaluates results and corrects course

The agent's cycle

Internally, an agent runs in a simple and powerful loop, repeated until the task is done:

20. **Think** — given the goal and current state, what's the next step?
21. **Act** — execute a tool (search the web, read a file, run code).
22. **Observe** — read the result of the action.
23. **Repeat** — go back to step 1 with the new information, until done.



The agent's cycle: think, act, observe — repeating until the task is ready.

KEY CONCEPT — Tools

What gives an agent superpowers are tools — functions it can call to interact with the world. Without tools, an agent only thinks. With them, it reads emails, queries databases, browses the internet and runs code. Much of building agents is giving them the right tools.

What an agent can do for you

- Read a spreadsheet, compute values and generate a formatted report
- Monitor an inbox and classify messages by priority
- Research a topic on the web, consolidate sources and write a summary
- Check a system, detect inconsistencies and alert when it finds something
- Coordinate other specialized agents for bigger tasks

CHAPTER 17

CrewAI: assembling your first team

CrewAI is the most intuitive framework for creating agents. The metaphor is brilliant: you assemble a crew of agents with roles, as if hiring specialized employees.

The CrewAI philosophy

Instead of programming the logic step by step, you describe who the agents are — role, goal and backstory — and the AI fills in the behavior. In 2026, CrewAI is an independent, lightweight framework, built from scratch (it doesn't depend on LangChain), on the 1.x line and focused on production reliability. It uses uv as its manager and supports any LLM, from local Ollama to Claude.

```
# Installation
pip install crewai

# The CLI creates a ready-made project structure
crewai create crew my_project
crewai version
```

Anatomy of an agent

Three fields define an agent, and each has a clear role:

- **role** — the job title. Defines the agent's perspective (e.g., "Financial Analyst").
- **goal** — the objective. What it seeks to achieve.
- **backstory** — the history. The context and specialty that shape how it acts.

```
from crewai import Agent, Task, Crew, Process

analyst = Agent(
    role='Payroll Analyst',
    goal='Calculate deductions accurately and follow company rules',
    backstory=('Payroll specialist fluent in tax and labor law. '
              'Methodical and detail-oriented, never rounds '
              'without a clear criterion.'),
    llm='ollama/qwen2.5-coder:14b', # local model
    verbose=True
)
```

TIP — Why the code is so "thin"

The intelligence isn't in your code — it's in the model. You only define the role; the LLM already knows, from its training, what a financial analyst does and how they think. It's like hiring someone experienced: you state the job and the mission, you don't teach the profession from zero.

Tasks and assembling the crew

Agents execute tasks. Each task has a description and an expected output. You gather agents and tasks into a crew and set it running.

```
task = Task(
    description='Calculate the social security deduction for a $3,000 salary',
    agent=analyst,
    expected_output='The deduction amount with the applied rate'
)

crew = Crew(
    agents=[analyst],
    tasks=[task],
    process=Process.sequential
)

result = crew.kickoff()
print(result)
```

WARNING — More agents, more tokens

Coordinating several agents passes context at each step. Community benchmarks indicate that multi-agent systems can consume about 4x more tokens than a single-agent approach. Monitor usage (`result.token_usage`) and don't use five agents where one will do.

CHAPTER 18

Tools: giving agents hands

An agent without tools is a brain in a jar. Tools are what connect it to the real world — files, internet, systems. Here's the catalog and how to use it.

Where tools come from

A common confusion: tools aren't "CrewAI's". CrewAI orchestrates them, but underneath they use specialized libraries. Understanding this separation helps with debugging and creating your own.

Tool (crewai_tools)	Real library underneath
ScrapeWebsiteTool	requests + BeautifulSoup
SerperDevTool	serper.dev search API
FileReadTool / FileWriterTool	Native Python (read/write)
CodeInterpreterTool	Python subprocess
RagTool	embeddings + vector database

Accessing files on the computer

```
from crewai_tools import FileReadTool, FileWriterTool, DirectoryReadTool

agent = Agent(
    role='File Assistant',
    goal='Read, organize and generate files',
    backstory='Specialist in document organization',
    tools=[FileReadTool(), FileWriterTool(), DirectoryReadTool()]
)
```

Browsing the internet

There are two levels. To search and read pages, search and scraping tools are enough. To truly interact — click, fill in, log in — you need to control a real browser.

```
from crewai_tools import SerperDevTool, ScrapeWebsiteTool

# Google search + page reading
researcher = Agent(
    role='Web Researcher',
    goal='Find and extract information from the internet',
    backstory='Meticulous digital investigator',
    tools=[SerperDevTool(), ScrapeWebsiteTool()]
)
```

NOTE — SerperDevTool needs a key

Google search via Serper requires a free API key from serper.dev — the free plan is generous. Set it as the environment variable `SERPER_API_KEY`.

Controlling a real browser

For automations that require clicking and filling (logging in, exporting reports from a web system), you use a browser automation layer. The `browser-use` library, based on Playwright, has become the reference: the agent literally sees the page and interacts like a human.

```
pip install browser-use
# It combines with the agent as a tool;
# it's the most powerful option and also the most complex.
```

WARNING — Be careful what the agent accesses

An agent that controls the browser and has your credentials can do damage if poorly instructed or tricked by malicious page content. Limit the scope, avoid giving access to sensitive accounts, and never let an agent perform irreversible actions without supervision.

CHAPTER 19

Custom tools: the real power

Ready-made tools are the start. The leap in capability comes when you realize that any Python code can become a tool — including access to your own systems.

Turning code into a tool

You create a class that inherits from `BaseTool`, describe what it does (the description is what the agent reads to decide to use it) and implement the `_run` method with any logic.

```
from crewai.tools import BaseTool

class PayrollLookupTool(BaseTool):
    name: str = 'Payroll Lookup'
    description: str = ('Looks up an employee's payroll data '
                        'by the given month.')

    def _run(self, month: str) -> str:
        # HERE goes any Python code:
        # access a database, internal API, spreadsheet,
        # legacy system... whatever you need.
        data = my_system.query(month)
        return str(data)
```

KEY CONCEPT — Why this is transformative

The agent can now use ANYTHING you can do in Python: your ERP, your database, a government API, an Excel spreadsheet, an old company system. The custom tool is the bridge between the model's intelligence and the real systems of your business.

The description is everything

The agent decides when to use a tool by reading its description. A vague description leads the agent to use the tool at the wrong time or ignore it. Be specific about what it does, what it takes and what it returns.

TIP

Write descriptions as if explaining to a new colleague: "Use this tool for X when you need Y. It takes Z and returns W." The clearer the description, the better the agent decides.

CHAPTER 20

Orchestration: processes and flows

With several agents, the question becomes: who does what, in what order? CrewAI offers coordination modes for different levels of complexity.

How agents communicate

Agents communicate through chained tasks: one's output becomes the next one's input. The context field connects a task to the result of another.

```
stock_task = Task(
    description='List products below the minimum stock level',
    agent=stock_clerk,
    expected_output='List of products to restock'
)

purchase_task = Task(
    description='Analyze the cost and approve the restock',
    agent=finance,
    expected_output='Approved list with budget',
    context=[stock_task] # receives the previous task's output
)
```

Sequential vs hierarchical mode

In sequential mode, tasks run in a fixed order, one after another. In hierarchical mode, a manager agent coordinates: it decides which agents to call, in what order, and can ask them to redo something unsatisfactory.

```
# Sequential: fixed order
crew = Crew(agents=[a, b, c], tasks=[t1, t2, t3],
            process=Process.sequential)

# Hierarchical: a manager coordinates
crew = Crew(agents=[a, b, c], tasks=[...],
            process=Process.hierarchical,
            manager_llm='ollama/qwen2.5-coder:14b')
```

TIP — Avoid delegation loops

In hierarchical mode, only the manager should have delegation enabled. If several agents can delegate to each other, they enter a loop passing tasks around. Set `allow_delegation=False` on the workers and limit `max_iterations`.

Flows: CrewAI's flow control

Recent versions brought Flows — a way to orchestrate steps with state, conditional routing and human-in-the-loop intervention points. While a crew is a team solving one task, a Flow is the larger script that chains crews, decisions and data across an entire process. For serious business automations, it's the structure that brings predictability and control.

KEY CONCEPT — Crew vs Flow

Use a Crew when agents need to collaborate on a task. Use a Flow when you have a process with several stages, conditions ("if this, do that") and state that persists. In real projects, Flows call Crews — the script coordinates the teams.

PART VI

Advanced Orchestration

LangGraph, the MCP protocol and the internet of agents being built right now

With one agent working, the question of scale arises: how to coordinate complex flows and connect agents to everything? This part shows the patterns and protocols the ecosystem adopted in 2026.

CHAPTER 21

LangChain and LangGraph

If CrewAI is the friendly front door, LangChain is the industry's complete toolbox. And LangGraph is what you use when flows get really complex.

LangChain: the Swiss Army knife

LangChain is the most widely adopted framework for building applications with LLMs. It offers standardized pieces to connect models to data, tools and flows. You already used it in this book, in the RAG chapter.

The core components

- **Chains** — sequences of model calls chained for composite tasks.
- **Retrievers** — components that fetch relevant documents (the basis of RAG).
- **Memory** — mechanisms to maintain context across interactions.
- **Tools and Agents** — callable functions and the logic that decides when to use them.

LangGraph: flows as graphs

LangGraph extends LangChain to model agents as state graphs: nodes (steps) connected by edges (transitions), with a state that travels through the flow. This unlocks what linear flows can't: loops, conditional branches, retries and multiple paths.

KEY CONCEPT — When the graph matters

Think of a support scenario: "if the customer is upset, escalate to a human; if the question is simple, answer; if it needs data, query the system and come back". That's a graph with decisions and loops — exactly what LangGraph models clearly, and where linear flows become a tangle of workarounds.

CrewAI vs LangGraph: which to choose

Criterion	CrewAI	LangGraph
Learning curve	Gentle	Steep
Simple to medium flows	Excellent	Can be overkill
Complex flows with decisions	Limited (but Flows help)	Ideal
State control	Abstracted	Explicit and total
Philosophy	Roles and collaboration	State machine

TIP

You don't have to pick one forever. Many start with CrewAI for its simplicity and adopt LangGraph when a specific flow demands fine control. The skills transfer — the logic of agents is the same.

LangSmith: seeing what the agent thinks

Debugging agents blindly is frustrating. LangSmith, from the LangChain team, offers a visual interface where you see every step: what the agent thought, which tools it called, what it received and why it decided the next move. It has a free plan for individual use and is almost indispensable when agents get non-trivial.

CHAPTER 22

MCP: the protocol that connects everything

In 2026, one standard became the foundation on which agents access the world. Understanding MCP is understanding where the whole ecosystem is heading.

The problem MCP solves

Before MCP, each connection between an agent and a tool was a custom integration. Connecting the AI to Google Drive, your database and Slack required three different integrations, each reinvented by every developer. Chaos, duplication and insecurity.

KEY CONCEPT — MCP — Model Context Protocol

Created by Anthropic, MCP is an open standard that defines ONE universal way for an agent to connect to tools, data and services. It's the "USB" of agents: instead of a different connector for each thing, a single standard everything speaks.

The architecture, in simple terms

MCP follows a client-server model using JSON-RPC. An MCP server exposes tools and data (for example, a server that gives file access, or to your database). An MCP client — the agent, or the editor — consumes those tools in a standardized way.

- **MCP servers** — expose capabilities: read files, query databases, automate the browser, access Drive.
- **MCP clients** — agents and tools that use those capabilities. Claude, Cursor, Claude Code and the main frameworks are MCP clients.

Why this matters to you

Adoption was extraordinary. MCP went from a novelty to the de facto standard in just over a year, with tens of millions of downloads and native support from the main labs (Anthropic, OpenAI, Google, Microsoft) and frameworks (LangChain, CrewAI, LangGraph, LlamaIndex). In December 2025, the protocol was donated to a neutral foundation (the Agentic AI Foundation, under the Linux Foundation), sealing its role as common infrastructure.

COST — What this means in practice

You're no longer locked to a vendor. Write an MCP server for your system once, and any agent — local or from any company — can use it. For Python, the `fastmcp` library has become the standard for building servers quickly.

Agents that browse the PC and the web via MCP

Back to a practical question: yes, agents can navigate the computer and the internet, and MCP is precisely what standardizes that access. There are MCP servers for browser automation, file systems, the terminal and dozens of services. Add Computer Use (a feature that lets the model see the screen and control mouse and keyboard) and you have agents capable of operating the computer like a human.

WARNING — Security first

Run MCP servers with file or network access in isolated environments (sandbox/Docker). Prefer read-only modes for untrusted agents and use scoped permissions. An agent with broad access is as dangerous as it is useful.

CHAPTER 23

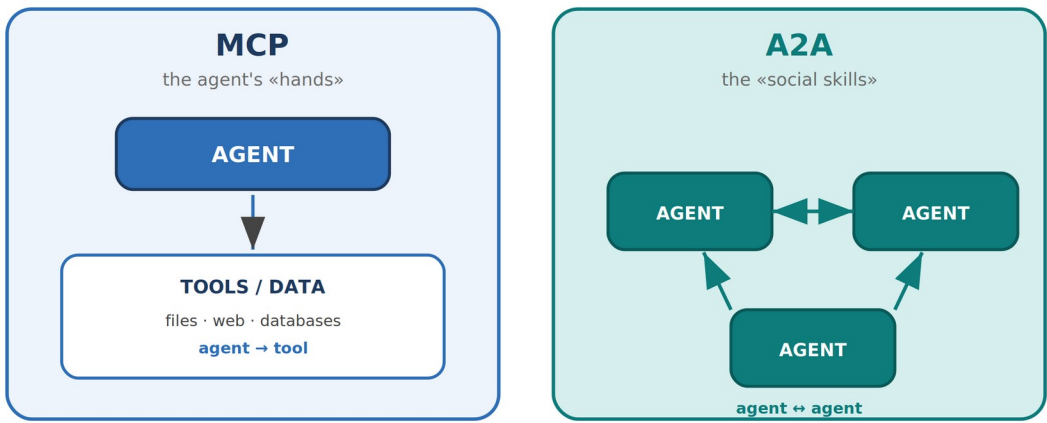
A2A and multi-agent systems

If MCP connects an agent to tools, another problem remains: how do agents from different origins talk to each other? That's where A2A comes in.

Two protocols, two roles

MCP and A2A are complementary, not competitors. The distinction is architectural and worth memorizing.

	MCP	A2A
Connects	Agent to tools/data	Agent to another agent
Focus	Execution and resource access	Negotiation and delegation
Metaphor	The agent's "hands"	The "social skills"
Origin	Anthropic	Google



MCP connects the agent to tools; A2A connects agents to each other. They are complementary.

KEY CONCEPT — The sentence to remember

MCP gives the agent the hands to touch the world. A2A gives the agent the ability to collaborate with other agents. In a sophisticated system, both operate at once: an orchestrator agent uses A2A to delegate subtasks to specialists, and each specialist uses MCP to execute.

Agents from different frameworks working together

A natural question: can an agent built in CrewAI call one built in LangGraph? Yes. There are two main ways. Via protocol (A2A), which standardizes discovery and delegation between agents. And via direct integration — exposing each agent as a tool or an endpoint that the others call.

```
from crewai.tools import BaseTool
```

```
class CallAccountingAgent(BaseTool):
    name: str = 'Accounting Agent'
    description: str = 'Accounting specialist (another framework)'

    def _run(self, data: str) -> str:
        # calls an external agent, from any framework
        return external_agent.invoke(data)

# The CrewAI manager can now delegate to the external agent
manager = Agent(role='Manager', goal='Coordinate specialists',
    backstory='Delegates to the right specialist',
    tools=[CallAccountingAgent()])
```

NOTE — CrewAI already speaks A2A

Recent CrewAI versions bring native A2A support as an install extra. The trend is for assembling heterogeneous agent teams — each in the framework it does best — to become increasingly plug-and-play.

When several agents are worth it

Multi-agent isn't always the answer. Each extra agent adds cost (tokens), latency and points of failure. Use several agents when the task has genuinely distinct parts that benefit from specialization and can be checked against each other. For most automations, a well-built agent with good tools is enough.

KEY CONCEPT — The rule of the smallest working system

Start with one agent. Add a second only when there's a clear division of responsibility that improves the result. Complexity is a cost, not a virtue.

PART VII

Fine-tuning

When and how to train a model for your specific domain

RAG gives the model knowledge; fine-tuning changes its behavior. This part explains the difference — and why, in most cases, you'll want to start with RAG.

CHAPTER 24

What fine-tuning is (and when you need it)

Fine-tuning sounds advanced and intimidating, and sometimes it is. But the concept is simple, and knowing WHEN to use it is more important than knowing how.

The idea

Fine-tuning is continuing the training of an existing model with your own data, teaching it to behave in a specific way. The model starts from everything it already knows and specializes in your case.

KEY CONCEPT — The specialization analogy

A base model is like a freshly graduated doctor: they know general medicine. Fine-tuning is the residency: they go into cardiology and become a specialist in that domain. In the end, they still know the general, but now they're an expert in the specific.

The decisive question: RAG or fine-tuning?

This is the most misunderstood point in the field. The two techniques solve different problems, and confusing them leads to months of wasted work.

You want the model to...	Solution
Know your documents and facts	RAG
Always answer in a format/style	Fine-tuning
Use your field's vocabulary and tone	Fine-tuning
Access data that changes often	RAG (just update the files)
Learn a very specific task	Fine-tuning
Get a fast and cheap result	RAG

TIP — The practical rule

RAG solves about 80% of business cases with a fraction of the effort. Always start there. Reserve fine-tuning for when you need to change the model's BEHAVIOR — not what it knows, but how it acts.

CHAPTER 25

LoRA, QLoRA and accessible fine-tuning

There was a time when training a model required GPU clusters and corporate budgets. Modern techniques brought fine-tuning within reach of a good personal machine.

Why LoRA changed the game

Training all the billions of parameters of a model is very expensive. LoRA (Low-Rank Adaptation) trains only a small fraction of new parameters, leaving the original model frozen. The result is almost as good as full training, at a fraction of the cost and time.

QLoRA adds quantization to the process, allowing large models to be trained on modest hardware — a 16GB GPU can fine-tune models that would otherwise require professional equipment.

- **Time advantage:** hours instead of days or weeks.
- **Hardware advantage:** runs on consumer GPUs.
- **Portable result:** generates a small file (the "adapter") that applies to the base model.

Tools for fine-tuning

Tool	Difficulty	Highlight
Unsloth	Accessible	Fast and efficient — recommended to start
LLaMA-Factory	Medium	Brings a web interface, supports many models
Axolotl	Medium/High	Very flexible, preferred by researchers
Hugging Face (TRL)	High	Industry standard, maximum control

The dataset is what matters

KEY CONCEPT — 90% of the result is in the data

The fine-tuning technique is nearly a commodity; what separates a good result from a bad one is the quality of the dataset. You need clean, consistent examples that represent the behavior you want to teach. A small, excellent dataset beats a large, messy one.

A fine-tuning dataset is, typically, a collection of input-output pairs in the format you want to teach. For an assistant that answers in a specific style, they're examples of questions and the ideal answers, written exactly as you want it to answer from then on.

WARNING — Fine-tuning doesn't memorize facts reliably

A common mistake is trying to "teach facts" via fine-tuning. The model learns patterns and style well, but it's not a reliable way to inject specific, updatable knowledge — for that, RAG is superior. Use fine-tuning for behavior, RAG for knowledge.

PART VIII

From Bench to Production

Costs, deployment, security and everything that separates an experiment from a reliable tool

Building something that works on your machine is half the journey. This part deals with the other half: what changes when it has to run for real, securely and with cost under control.

CHAPTER 26

Costs: local vs API, no illusions

The decision between running locally and using an API is, at heart, about costs and trade-offs. Let's put the numbers on the table.

The two cost models

Local model: you pay once for the hardware and then only for electricity. Marginal cost per use: practically zero. API model: you don't buy hardware, but you pay per token on every call. Marginal cost: proportional to usage.

Anthropic API prices (verified in 2026)

The prices below are per million tokens (MTok), billed separately for input and output. Output generation costs five times the input across all current-generation models.

Model	Input (per MTok)	Output (per MTok)	Profile
Claude Haiku 4.5	US\$ 1.00	US\$ 5.00	Fast, high volume
Claude Sonnet 4.6	US\$ 3.00	US\$ 15.00	Best balance
Claude Opus 4.8	US\$ 5.00	US\$ 25.00	Maximum capability

COST — Putting it in perspective

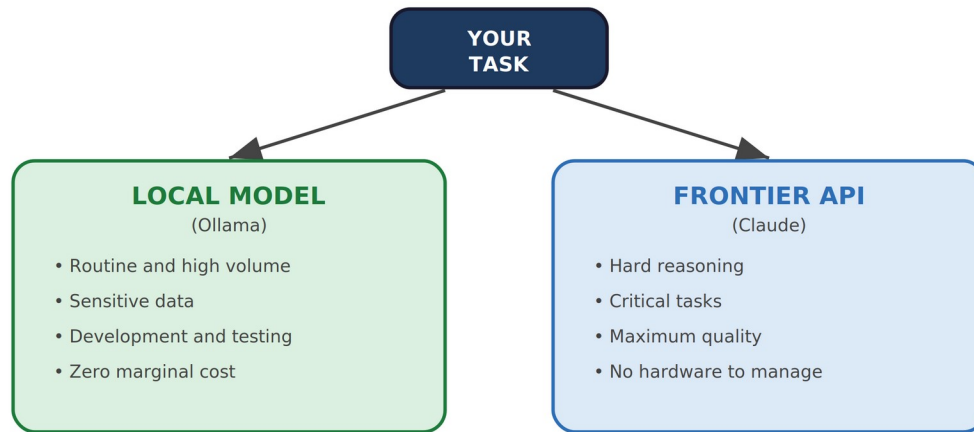
Current models include a 1M-token context window at the standard price. Features like prompt caching (up to 90% off repeated inputs) and batch processing (50% off) cut the bill significantly under real workloads. A well-optimized application with Sonnet usually costs tens of dollars per month for moderate traffic.

NOTE — App plan vs API

When you use Claude in an agent, the cost goes through the Anthropic API (console.anthropic.com), billed per token — not through your claude.ai app subscription. They're separate products: the subscription is for personal use in the interface; the API is for what you build.

The hybrid strategy

The smartest approach is rarely "all local" or "all API". It's to combine.



The hybrid strategy: local model for the routine, sensitive and high-volume; frontier API for the hard and critical.

- **Development and testing:** local model — iterate freely without spending a cent on API.
- **Repetitive, high-volume tasks:** local model — the zero marginal cost pays off.
- **Sensitive data:** local model, mandatory.
- **Hard, critical reasoning:** top-tier API (Sonnet/Opus) — worth the cost where quality matters.

CHAPTER 27

Deployment: getting the agent to run 24/7

An agent on your PC only works while the machine is on. For it to work on its own, day and night, it needs to live on a server. Here's how to think about it.

The question that defines everything: where does the model run?

The choice of server depends entirely on where inference happens. This is the most common mistake for beginners: renting a weak server to run a heavy local model.

Agent architecture	Required server
Agent + API (Claude/GPT)	Basic, cheap VPS — the weight is in the AI's cloud
Agent + local 7-8B model	VPS with 16GB+ of RAM
Agent + local 14B+ model	Server with a GPU or lots of RAM
Personal use, not 24/7	Your own computer

KEY CONCEPT — The orchestrating agent is light

If your agent uses an external API for the reasoning, the agent itself barely consumes resources — it only coordinates calls and moves data. In that case, a VPS costing a few dollars a month runs fine, 24 hours a day. The heavy processing is on the AI's servers, not on your VPS.

WARNING — Weak VPS + local model = frustration

Trying to run a 7B model on a 2GB-RAM VPS results in crashes and errors. If the model runs on the machine itself, it needs muscle. There's no shortcut for that.

The recommended combination to start

For your first agent in production, the best cost-simplicity ratio is usually: a cheap VPS running the agent, which calls the Claude API for the reasoning. You pay little for the machine, you don't manage any GPU, and you get continuous availability. When volume grows to the point where the API bill hurts, then it's worth considering migrating to a local model on dedicated hardware.

CHAPTER 28

Security, privacy and supervision

Agents with the power to act in the world carry risks proportional to their power. This is the part many skip — and shouldn't.

The real risks of autonomous agents

- **Irreversible actions:** an agent that deletes files, sends emails or moves money can cause damage that can't be undone.
- **Instruction injection:** malicious content on a page or document can "hijack" an agent, making it follow orders hidden in the content.
- **Data leakage:** a misconfigured agent can send sensitive information where it shouldn't.

WARNING — The golden rule of agents

Content the agent reads (pages, emails, documents) is DATA, not COMMANDS. Never let an agent execute instructions it found in external content without human confirmation. Asking an agent to "handle the to-do list" authorizes reading the list — not blindly executing whatever is written in it.

Defense principles

24. **Least privilege:** give the agent only the access strictly necessary. Read-only tools whenever possible.
25. **Human in the loop:** for irreversible actions (send, publish, delete, pay), require human confirmation.
26. **Isolation:** run agents with system access in isolated environments (sandbox, Docker).
27. **Logging and auditing:** log everything the agent does. When something goes wrong — and it will — you need to know what and why.

The privacy advantage of local

KEY CONCEPT — Why sensitive data calls for a local model

When you process personal IDs, salaries, contracts or health data, sending them to an external API may violate internal policies and data protection laws. A local model solves this at the root: the data never leaves your infrastructure. For many businesses, that's the number-one reason to run AI locally.

PART IX

Getting Serious

Prompting, API and quality — the bridge between a working prototype and a reliable product

You already know how to build. This part is about building well: getting the most out of each model with good prompts, connecting to the API when it's worth it, and — the step almost everyone skips — testing quality.

CHAPTER 29

The art of the prompt

This is perhaps the highest-return skill in the whole book. The same model, on the same hardware, delivers radically different results depending on how you talk to it. A good prompt is half the work.

Why the prompt matters so much

An LLM does exactly what the input text leads it to do. It doesn't guess your intention — it continues the pattern you started. A vague prompt produces a vague answer; a precise prompt produces a precise answer. The good news is that improving prompts requires no technical knowledge, only method.

KEY CONCEPT — The model doesn't read your mind

Everything the model knows about what you want is in the prompt. Context that's clear in your head but absent from the text simply doesn't exist for it. Writing a good prompt is, largely, making explicit what was implicit.

The four pillars of a good prompt

1. Clarity and specificity

Say exactly what you want. Compare:

BAD: Talk about payroll.

GOOD: Explain, in 3 paragraphs and plain language, how the social security deduction works for a full-time employee, for someone with no technical background.

2. Context

Give the model the background it needs. Who you are, who the answer is for, what the situation is. The same request changes completely depending on the context provided.

3. Examples

Showing is stronger than telling. If you want a specific format, give an example of it. This technique, called few-shot, is so powerful it has its own chapter (30).

4. Output format

Specify how you want the answer: list, table, JSON, number of paragraphs, tone. If you don't say, the model chooses for you — and not always how you'd like.

TIP — The structure that almost never fails

Role + Task + Context + Format. "You are a [role]. Do [task]. Consider that [context]. Answer in [format]."
This simple recipe handles most everyday cases.

The system prompt

Besides your question, most tools allow a system prompt: general instructions that apply to the whole conversation. It's where you define the personality, rules and limits of the assistant — once, instead of repeating in every message.

```
# Example system prompt for an HR assistant
You are an HR assistant. Always answer based only on the
provided policies. Use a friendly, clear tone.
If you don't know, say you'll forward it to the right team.
Never invent values or deadlines.
```

Iteration: the secret to good results

The first prompt is rarely the best. The professional flow is iterative: write, see the result, identify what was missing, and refine. Each adjustment teaches you something about how that model "thinks". Treat the prompt as a draft, not as stone.

WARNING — Different models, different prompts

A prompt tuned for Claude may perform worse on a local 8B model, and vice versa. Smaller models need more explicit instructions and more detailed examples. When you switch models, expect to readjust your prompts.

CHAPTER 30

Advanced prompting techniques

With the fundamentals in place, a few techniques raise quality to another level — especially in hard, reasoning, or structured-output tasks.

Few-shot: teaching by example

Instead of describing what you want, show a few finished examples. The model captures the pattern and reproduces it. It's the most effective technique for ensuring consistent format and style.

```
Classify the sentiment of each sentence as POSITIVE,  
NEGATIVE or NEUTRAL.
```

```
Sentence: The service was excellent! -> POSITIVE  
Sentence: I waited two hours in line. -> NEGATIVE  
Sentence: The product arrived Tuesday. -> NEUTRAL  
Sentence: I loved it, highly recommend! ->
```

The model completes the pattern and answers POSITIVE. Two or three well-chosen examples usually suffice.

Chain-of-thought: think before answering

For problems that require reasoning (logic, math, multi-step decisions), ask the model to think step by step before giving the final answer. This drastically reduces errors, because the model "shows its work" instead of jumping to a conclusion.

```
Solve the problem by thinking step by step before  
giving the final answer.
```

```
Problem: An employee earns $4,000 and has an 8%  
social security deduction and 5% transit benefit.  
What is the net amount?
```

KEY CONCEPT — Why it works

Models generate text sequentially. When they "reason out loud", each correct step becomes the basis for the next. Forcing explicit reasoning before the answer leverages that mechanism in your favor. Reasoning models (like the DeepSeek-R1 derivatives) do this automatically.

Structured output (JSON)

When the result will feed another program, you need a predictable format, not prose. Ask explicitly for JSON and describe the fields. This lets you integrate AI into real systems.

```
Extract the data from the text and answer ONLY with JSON  
in this format, with no text before or after:  
{ "name": "", "role": "", "salary": 0 }
```

```
Text: Mary Smith is an analyst and earns $5,500 per month.
```

TIP — Ensuring valid JSON

For serious integration, ask for "only the JSON, no markdown" and, in code, handle the response carefully: strip any code fences (```) before parsing. Many libraries now offer structured-output modes that guarantee valid JSON — worth using.

Decomposition: breaking down the problem

Complex tasks come out better when divided. Instead of asking for everything at once, break it into steps — whether in a single prompt with numbered steps, or in chained calls where one's output feeds the next. It's the same principle as agents, applied to the prompt.

CHAPTER 31

Using the Claude API in your code

Local models cover a lot, but there are tasks where a top-tier model's quality pays off. Connecting your code to Claude is simple — and it's the other side of the hybrid strategy.

First steps

You need an API key, obtained from the Anthropic console (console.anthropic.com). Billing is per token, separate from the app subscription. Store the key in an environment variable — never write it directly in the code.

```
pip install anthropic

# Set the key as an environment variable (in the terminal):
export ANTHROPIC_API_KEY='your-key-here'
```

A basic call

```
import anthropic

client = anthropic.Anthropic() # reads the key from the environment

response = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=1000,
    messages=[
        {'role': 'user', 'content': 'Explain RAG in one paragraph.'}
    ]
)
print(response.content[0].text)
```

NOTE — Choosing the model

Use `claude-haiku-4-5` for simple, high-volume tasks (cheaper and faster), `claude-sonnet-4-6` as a balanced default, and `claude-opus-4-8` when you need maximum capability. The names evolve — confirm the current identifiers in the official documentation.

Connecting Claude to a CrewAI agent

Switching an agent's model from local Ollama to Claude is one line. That's the essence of the hybrid strategy: cheap agents running locally, and the critical ones on Claude.

```
from crewai import Agent
```

```
# Before: local model
# llm='ollama/qwen2.5-coder:14b'

# After: Claude for the task that demands maximum quality
analyst = Agent(
    role='Senior Analyst',
    goal='Produce the final analysis with maximum rigor',
    backstory='An expert who makes no slips',
    llm='anthropic/claude-sonnet-4-6'
)
```

COST — Controlling spend

Set `max_tokens` sensibly — long output costs 5x the input. Use prompt caching for repeated content (up to 90% off) and batch processing when you don't need an immediate response (50% off). And reserve Opus for when Sonnet can't handle it: the price difference is large.

System prompt in the API

In the API, the system prompt goes in its own parameter, separate from the messages. It's where you fix the assistant's behavior.

```
response = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=1000,
    system='You are a rigorous technical reviewer. Point out errors '
          'precisely and suggest objective fixes.',
    messages=[{'role': 'user', 'content': text_to_review}]
)
```

CHAPTER 32

Evaluating and testing AI systems

This is the part that separates a toy from a reliable tool — and the one most often ignored. If you're going to depend on an AI system, you need to know, with method, whether it works.

The problem with eyeballing it

It's tempting to test an agent by asking two or three questions, seeing it answer well, and declaring it done. The danger: you tested the easy cases. AI systems fail precisely on the edge cases — the spreadsheet with a strange format, the ambiguous question, the poorly scanned document. Without systematic testing, those errors only show up in front of the customer.

KEY CONCEPT — The test set rule

Build a fixed set of test cases — real inputs with the correct answers known. Every time you change a prompt, a model or a setting, run the whole set. That way you measure whether you really improved, instead of trusting your impression.

What to measure

- **Accuracy:** is the answer factually correct? Compare with your test set's answer key.
- **Source fidelity (in RAG):** is the answer really based on the documents, or did the model invent?
- **Consistency:** does the same question yield equivalent answers across repeated runs?
- **Robustness:** does it handle strange, incomplete or malicious inputs well?
- **Cost and latency:** how much time and how many tokens does each answer consume?

The "AI judge" technique

Evaluating hundreds of answers by hand is unfeasible. A practical technique is to use a strong model (like Claude) to evaluate another's outputs, according to criteria you define. It's not perfect, but it scales evaluation in a surprisingly useful way.

```
criteria = '''Rate the answer from 0 to 10 on:
- fidelity to the provided documents
- clarity and objectivity
Answer only with the number and a one-line justification.'''

# Send the question, the documents, the generated answer
# and the criteria to a strong model to evaluate.
```

Observability tools

For production systems, tools like LangSmith (seen in Chapter 21) log every run, letting you inspect steps, measure costs and build evaluation sets from real cases. In production, observing is as important as building.

WARNING — Before selling or delivering

If this material — or any system you build with it — goes into someone else's hands, test it rigorously. Run your set of cases, verify the numbers against official sources, and confirm the code works in the customer's environment. The trust you build with quality is the hardest asset to recover once lost.

PART X

Hands-on Projects

Three complete systems for you to adapt — from theory to what actually runs

Time to bring it all together. These three projects combine the book's concepts into systems you can copy, adapt and put to work right away.

CHAPTER 33

Project 1: Document assistant (RAG)

The first project solves a universal problem: having an AI that knows your company's documentation and answers questions based on it, citing the source.

The goal

Build an assistant that indexes a set of documents (manuals, policies, rules) and answers questions based exclusively on them. Everything running locally, with no data leaving the machine.

The architecture

- **Source:** a folder with the company's PDFs and texts.
- **Indexing:** local embeddings (nomic-embed-text) in a Chroma database.
- **Brain:** a local model via Ollama, with low temperature.
- **Interface:** AnythingLLM for the no-code version, or the script below for the programmable one.

Quick version (no code)

28. Install AnythingLLM and configure Ollama as the LLM and embedding provider.
29. Create a workspace, e.g., "HR Knowledge Base".
30. Drag all the relevant documents into the workspace.
31. In the workspace's system prompt, instruct it to answer only based on the documents and to clearly state when the information isn't available.
32. Done — ask and get grounded answers, with the sources.

Programmable version

Reusing the pipeline from Chapter 14, with an instruction that reduces hallucination and a question loop in the terminal:

```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import OllamaEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_community.llms import Ollama
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

# Load every PDF in a folder
loader = DirectoryLoader('docs/', glob='**/*.pdf', loader_cls=PyPDFLoader)
docs = loader.load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=150).split_documents(docs)
```

```

db = Chroma.from_documents(
    chunks, OllamaEmbeddings(model='nomic-embed-text'))

# Instruction that forces fidelity to the source
prompt = PromptTemplate.from_template(
    'Answer ONLY based on the context below. If the answer '
    'is not in the context, say you could not find it.\n\n'
    'Context: {context}\n\nQuestion: {question}\nAnswer:')

rag = RetrievalQA.from_chain_type(
    llm=Ollama(model='qwen2.5-coder:14b', temperature=0),
    retriever=db.as_retriever(search_kwargs={'k': 4}),
    chain_type_kwargs={'prompt': prompt})

while True:
    q = input('\nQuestion (or "quit"): ')
    if q.lower() == 'quit': break
    print(rag.invoke(q)['result'])

```

TIP — How to adapt it to your case

Swap the docs/ folder for yours. For documents that change, reindex periodically. For large volume, swap Chroma for Qdrant. For maximum answer quality, scale up the model (or point to the Claude API for the hardest questions).

CHAPTER 34

Project 2: Office automation agent

The second project enters agent territory: a system that reads data, applies rules and produces a result — automating a genuinely repetitive task.

The goal

An agent that receives data (for example, an employee spreadsheet), consults business rules via RAG, runs the calculations and generates a report. It's the reusable skeleton of countless office automations.

WARNING — About sensitive calculations

This example is didactic. Payroll, tax and legal calculations must be validated by a professional and against an up-to-date official source. Use the agent to organize and speed up the work — not to replace human review in legal or financial decisions.

The architecture

- **Reader agent:** reads the input spreadsheet with a file tool.
- **Rules tool (RAG):** consults the rules document for each case.
- **Analyst agent:** applies the rules and organizes the results.
- **Reporter agent:** generates the final report to a file.

The skeleton in CrewAI

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import FileReadTool, FileWriterTool, RagTool

rules = RagTool(config={'pdf': 'internal_rules.pdf'})

reader = Agent(
    role='Data Reader',
    goal='Extract and structure the data from the input spreadsheet',
    backstory='Meticulous with tabular data',
    tools=[FileReadTool()], llm='ollama/qwen2.5-coder:14b')

analyst = Agent(
    role='Analyst',
    goal='Apply the internal rules to each record',
    backstory='Knows the policies and consults the documentation',
    tools=[rules], llm='ollama/qwen2.5-coder:14b')

reporter = Agent(
    role='Reporter',
```

```
goal='Generate a clear report with the results',
backstory='Communicates results clearly',
tools=[FileWriterTool()], llm='ollama/qwen2.5-coder:14b')

t1 = Task(description='Read data.csv and structure the records',
          agent=reader, expected_output='Structured records')
t2 = Task(description='Apply the rules to each record',
          agent=analyst, expected_output='Calculated results',
          context=[t1])
t3 = Task(description='Generate report.md with the results',
          agent=reporter, expected_output='File generated',
          context=[t2])

crew = Crew(agents=[reader, analyst, reporter],
           tasks=[t1, t2, t3], process=Process.sequential)
print(crew.kickoff())
```

KEY CONCEPT — The reusable pattern

Reader -> Analyst (with a rules RAG) -> Reporter is a template that fits dozens of automations: invoice reconciliation, request triage, compliance checking, standardized document generation. Swap the source, the rules and the output format, and you have a new system.

CHAPTER 35

Project 3: Autonomous multi-agent researcher

The third project combines everything: agents that browse the web, specialists that collaborate, and a consolidated final product. It's the demonstration of what the ecosystem makes possible today.

The goal

A system that, given a topic, researches the internet, critically analyzes the sources and produces a structured report saved to a file. A researcher that works on its own from start to finish.

The architecture

- **Researcher:** uses web search and scraping to gather material.
- **Analyst:** evaluates the quality and relevance of the sources, separating signal from noise.
- **Writer:** consolidates everything into a clear, well-structured report.

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, ScrapeWebsiteTool, FileWriterTool

researcher = Agent(
    role='Web Researcher',
    goal='Gather the most relevant information about the topic',
    backstory='Digital investigator who cross-checks multiple sources',
    tools=[SerperDevTool(), ScrapeWebsiteTool()],
    llm='ollama/qwen2.5-coder:14b')

analyst = Agent(
    role='Critical Analyst',
    goal='Assess the credibility and relevance of the sources',
    backstory='Healthily skeptical, discards weak sources',
    llm='ollama/qwen2.5-coder:14b')

writer = Agent(
    role='Writer',
    goal='Write a structured and clear report',
    backstory='Communicator who organizes complex ideas',
    tools=[FileWriterTool()], llm='ollama/qwen2.5-coder:14b')

topic = 'AI automation trends for small and medium businesses'
t1 = Task(description=f'Research about: {topic}',
           agent=researcher, expected_output='Raw material gathered')
t2 = Task(description='Filter and assess the gathered sources',
           agent=analyst, expected_output='Synthesis of the best sources',
           context=[t1])
t3 = Task(description='Write a structured report.md',
```

```
agent=writer, expected_output='Final report in a file',
context=[t2])

crew = Crew(agents=[researcher, analyst, writer],
            tasks=[t1, t2, t3], process=Process.sequential)
crew.kickoff()
```

TIP — Leveling up

For professional results, swap the LLM of the critical agents (analyst and writer) for the Claude API — the quality of reasoning and writing takes a leap. Keep the researcher local to save on the many gathering calls. It's the hybrid strategy from Chapter 26 applied in practice.

From project to product

These three projects are starting points. The path to turning them into something of real value is always the same: pick a concrete, repetitive problem from your daily routine, adapt the closest template, and iterate. Knowledge of the problem — which you have and most technical people don't — is what turns a generic script into a solution worth money.

KEY CONCEPT — Your competitive edge

Most developers know how to program agents, but don't know the real problems of specific domains. Those who combine business knowledge with these tools build solutions that truly matter. That combination is rare — and valuable.

PART XI

Appendices

Quick reference to consult whenever you need

Keep these pages close. This is the material you'll reopen many times — terms, commands and fixes for the most common stumbles.

CHAPTER A

Glossary

The book's essential terms, gathered in one place. Consult it whenever you hit something unfamiliar.

Term	Definition
A2A	Protocol (Google) for agents of different origins to talk to each other.
Agent	AI that takes autonomous actions to achieve a goal, using tools.
API	A way for a program to access a service (e.g., Anthropic's AI) through code.
Chunk	A piece a document is split into for indexing in RAG.
Context	The amount of tokens the model can see at once.
CrewAI	Framework for assembling teams of agents with roles.
Embedding	A numeric vector that represents the meaning of a text.
Fine-tuning	Continuing the training of an existing model to specialize it for your case.
Flow	In CrewAI, the script that chains steps, decisions and crews into a process.
Hallucination	When the model states something false with the appearance of truth.
IDE	Integrated development environment (editor + tools).
Inference	The act of the model generating a response from a prompt.
LangChain	Complete framework for applications with LLMs.
LangGraph	Extension of LangChain for complex flows as state graphs.
LLM	Large language model (Claude, GPT, Llama, Qwen...).
LoRA / QLoRA	Efficient fine-tuning techniques that train only part of the model.
MCP	Protocol (Anthropic) that standardizes agents' access to tools.
Ollama	Tool for running open models locally.
Parameters (B)	Billions of learned values; they measure the model's capacity.
Prompt	The input text sent to the model.
Quantization	Compressing a model by reducing precision, to fit in memory.
RAG	Retrieving chunks of documents before generating the answer.
Temperature	Controls how creative (high) or precise (low) the answer is.
Token	A piece of text; the unit of measurement and billing for models.
Tool	A function an agent can call to act in the world.
Vector database	Database optimized to search by similarity of meaning.
VRAM	Graphics card memory; it defines which model fits on the GPU.

CHAPTER B

Command reference

The most-used commands, in one place, to copy when you need them.

Ollama

```
ollama run <model>           # downloads and starts the chat
ollama run <model> --verbose  # shows tokens/s
ollama pull <model>          # only downloads
ollama list                  # installed models
ollama ps                   # what's running and on which hardware
ollama show <model>         # spec sheet and context
ollama rm <model>           # removes from disk
```

Python environment

```
python -m venv .venv          # creates a virtual environment
source .venv/bin/activate     # activates (Linux/Mac)
.venv\Scripts\activate        # activates (Windows)
pip install <package>         # installs a library
pip install uv                # modern manager (optional)
```

Main installs

```
pip install crewai crewai-tools
pip install langchain langchain-community langgraph
pip install langchain-anthropic # to use Claude
pip install chromadb pypdf      # RAG
pip install browser-use         # web browsing
ollama pull nomic-embed-text    # embeddings for RAG
```

Claude Code

```
npm install -g @anthropic-ai/claude-code # requires Node 18+
claude                                    # starts in the folder
claude --version                          # version
claude doctor                             # diagnostics
```

CHAPTER C

To keep learning

The field moves fast. These sources help you keep your knowledge current after this book.

Resource	Type	Where to find it
Ollama documentation	Official	ollama.com
CrewAI documentation	Official	docs.crewai.com
LangChain documentation	Official	python.langchain.com
MCP documentation	Official	modelcontextprotocol.io
Hugging Face	Platform	huggingface.co
Unsloth (fine-tuning)	Repository	github.com/unslothai/unsloth
r/LocalLLaMA	Community	Reddit — local AI

KEY CONCEPT — The habit that matters most

No book keeps up with the pace of this field — not even this one. The most valuable skill you can cultivate is checking the current state before deciding: confirming prices, versions and commands against official sources. Those who do it build on solid ground; those who trust memory build on stale data.

CHAPTER D

Common troubleshooting

The obstacles almost everyone hits, and how to get past them.

The model is very slow

Check with ollama ps whether it's 100% on the GPU. If there's a share on CPU, the model didn't fit in VRAM — use a smaller one or a more aggressive quantization. Close programs that consume video memory.

Permission error when installing with npm

Never use sudo. Configure your own directory: npm config set prefix '~/.npm-global' and add that path to your PATH.

The libraries conflict with each other

It's almost always a missing virtual environment. Create one per project with python -m venv and activate it before installing anything.

The agent goes into a loop

In hierarchical mode, make sure only the manager has delegation. Set allow_delegation=False on the workers and limit max_iterations to a reasonable value (e.g., 10).

RAG answers poorly or makes things up

Review three things: the quality of the source documents, the chunk size (neither too large nor too small) and the value of k. Add to the prompt the instruction to answer only based on the context, and lower the temperature.

The API cost is high

Multi-agent systems consume many tokens. Monitor usage, reduce the number of agents, use a cheaper model (Haiku/Sonnet) where possible, and move repetitive tasks to a local model.

From theory to practice

You've reached the end of the content, but the beginning of what matters: doing. All of this material only becomes real knowledge when it meets a keyboard. Pick a small, real problem from your daily routine, open the terminal, and build the first version — imperfect, simple, working. The rest you learn by doing.

The best time to start is now.